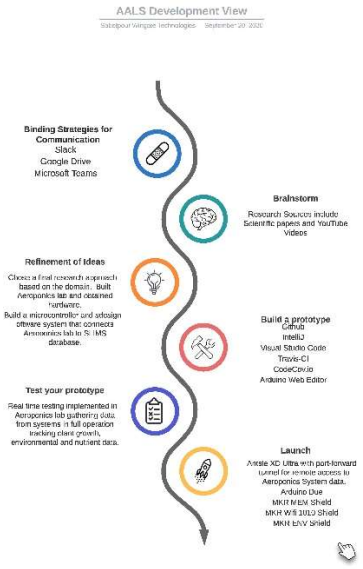
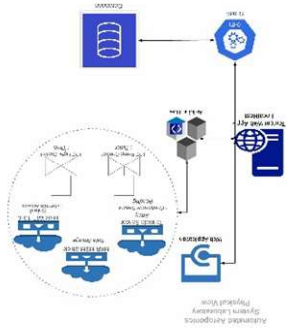
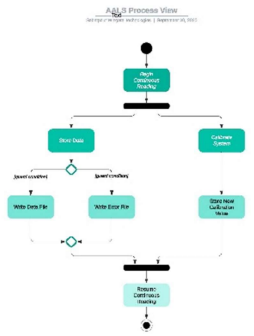
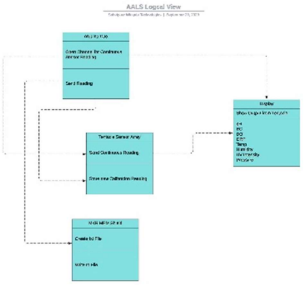


The Architecture of the AALS

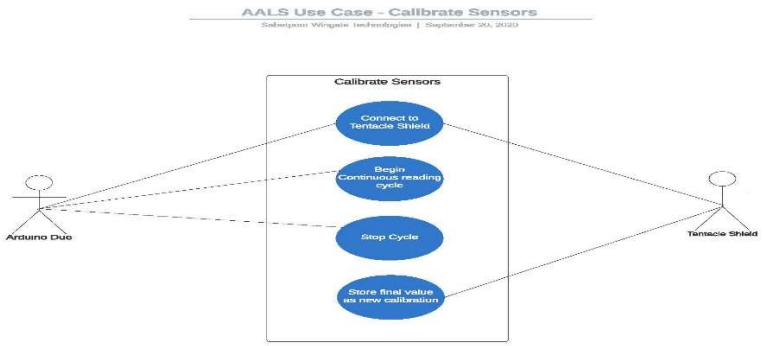
Model-driven software engineering was implemented in the architecting of the AALS. We used the 4+! Architecture view to develop baseline models from the perspectives of the development, locgical, physical and process views. In addition, we created a preliminary use case for the System Calibration functionality of the micro-controller. This MDSE approach informed our decision to engage iterative cycles of prototyping in our product development lifecycle. At the beginning of this process, models provided a high-level overview but with each consecutive iteration, added insight leant complexity to the prototypes developed from the models. Our first models allowed us to develop prototypes for simple data capture from a POJO class. Our final models allowed us to capture data from the POJO class and parse it into specific variables to be displayed in the UI on a grid template.







In Domain Driven Design by Eric Evans, there is a discussion of the idea of conceptual contours. These are depicted as divisions within the domain. These divisions are intended to be highly cohesive while simultaneously remaining highly independent. Such fluidity between divisions facilitates more efficient refactoring within the domain and greater stability within the system in direct response to refactoring. Conceptual contours are a characteristic necessity of supple design in architecture. Architecting the AALS reflects this approach via the implementation of standalone classes within the module. As an example, the slimsgate plugin was designed with a collection of handlers that are called by the route builder bean. Each of these classes runs independently and to completion prior to the call for the next bean on the route. As Evans explains, this is an exceedingly useful pattern in supple design as it results in an overall reduction of interdependency and lends a richness to the system.



Strategy is a key component of supple design. Knowing the context, or limits of the model is imperative to the consistency of the tested performance of the prototypes conceived by the model. Unambiguous function of the conceptual contours through expediency of the rules, eliminates overlap and contradiction within the interface. A unified model is the infrastructure upon which a prototype should ideally be built. The context in which our AALS slimsgate plugin was architected began with REST API HTTP post request and evolved to include a restlet endpoint which initiated the route builder and subsequently scheduled activities utilized to parse and on store the data sent from the microcontroller.